

Module Title and Code: Artificial Intelligence for Cyber Security (WM9PH-15)

Component Type: Written Report

Student ID: 5746897

1. Analyse the Problem and Dataset

This project applies supervised machine learning to the CIC-IDS2017 MachineLearningCVE dataset for network intrusion detection. CIC-IDS2017 is a recognised intrusion detection benchmark containing labelled benign and attack traffic generated from realistic user profiles and network attack scenarios (Sharafaldin, Habibi Lashkari and Ghorbani, 2018; Canadian Institute for Cybersecurity, 2017). The task is formulated as binary classification: each network flow is classified as either BENIGN or ATTACK. All original non-BENIGN labels are grouped into the ATTACK class. This means the model is designed to detect whether traffic is malicious, rather than identifying the exact attack type.

Binary classification was chosen because the original dataset is highly imbalanced across attack categories. Some attack classes contain many examples, such as DoS Hulk (172,846), DDoS (128,014) and PortScan (90,694), while others are extremely rare. In the final deduplicated modelling dataset, there were only 36 Infiltration records, 21 Web Attack Sql Injection records and 11 Heartbleed records. These very small classes would make reliable multi-class attack prediction difficult within the scope of this coursework. Grouping all malicious labels into ATTACK therefore gives a more stable intrusion-detection task while still using the full dataset. The imbalance across the original attack categories is shown in Figure 1.

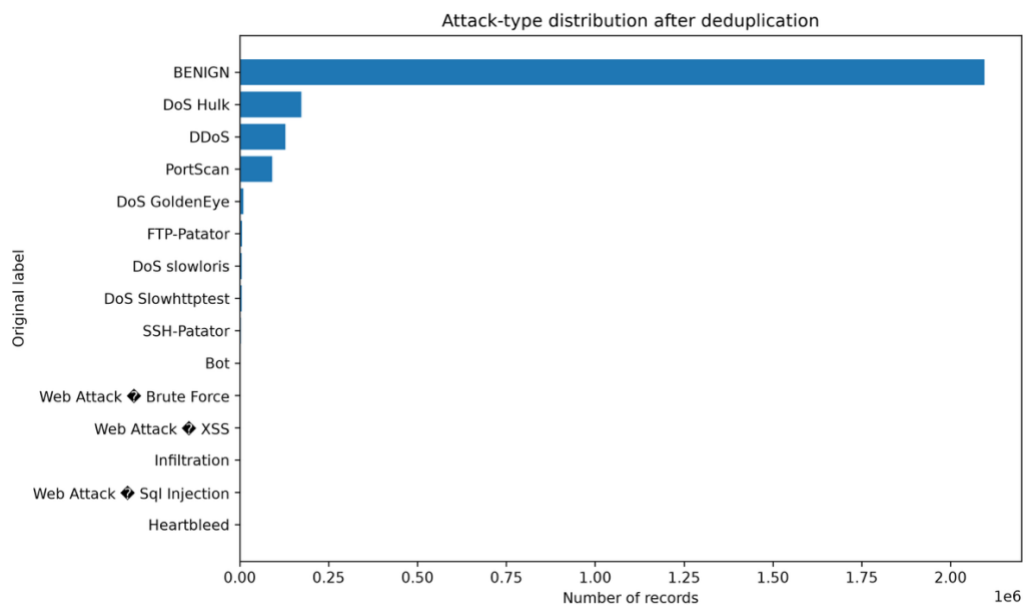


Figure 1. Attack-type distribution in the deduplicated modelling dataset.

The raw dataset contained eight CSV files representing different capture days and attack scenarios. After combining the files, the dataset contained 2,830,743 rows and 81 columns. Infinite values were replaced with missing values, and rows containing missing values were removed. The only affected columns were Flow Bytes/s and Flow Packets/s, each with 2,867 missing values. Removing these rows reduced the dataset to 2,827,876 rows. This cleaning step was necessary because leaving infinite or missing flow-rate values could affect model training and metric reliability.

An important data-quality issue was the presence of exact duplicate records. These were removed before splitting the dataset to reduce the risk of memorisation between training and test data. In total, 307,078 duplicate rows were removed, which was 10.859% of the cleaned dataset. This was an important step because previous research has identified errors and quality issues in CICIDS2017 that can significantly affect detection performance (Lanvin et al., 2023). After deduplication, the final modelling dataset contained 2,520,798 rows. The binary class distribution was still imbalanced, with 2,095,057 BENIGN records and 425,741 ATTACK records. This corresponds to 83.11% BENIGN and 16.89% ATTACK.

This imbalance affects the choice of evaluation metrics. Accuracy alone is not sufficient because a model could achieve a high score by favouring the majority BENIGN class. Therefore, the evaluation focuses on attack precision, attack recall, attack F1, macro F1, PR-AUC and confusion matrices in addition to accuracy. Attack recall is important because false negatives represent malicious traffic missed by the detector, while attack precision is important because false positives can create unnecessary alerts in a real security monitoring environment.

The dataset also has a structural limitation. CIC-IDS2017 is organised by capture day and attack scenario, meaning that a random split can place very similar traffic patterns from the same day on both sides of the train/test split. Even after exact duplicate removal, near-duplicate or capture-specific flow patterns may remain. For this reason, this project uses a random stratified split as the main in-distribution benchmark but also includes a stricter held-out Friday experiment to test whether the models generalise to different capture-day and attack-scenario conditions.

2. Model Selection and Comparative Analysis

Several supervised learning models were considered for the binary intrusion detection task. A Dummy Classifier was first used as a baseline to show the minimum performance expected from a model that does not learn meaningful traffic patterns. Logistic Regression, Decision Tree, Random Forest, Extra Trees and HistGradientBoosting were then compared on the validation set. These models were implemented using Scikit-learn, a widely used Python machine learning library (Pedregosa et al., 2018).

Logistic Regression was included as a simple linear model. It estimates the probability that a flow belongs to the ATTACK class using a weighted combination of the input features. Its main advantages are interpretability, speed and lower tendency to overfit compared with more complex models. It also provides probability scores, which makes threshold tuning possible. However, its limitation is that it assumes a mostly linear decision boundary. Network intrusion data often contains non-linear relationships between flow duration, packet lengths, ports, packet rates and TCP flags, so Logistic Regression may underfit complex attack patterns.

Tree-based models were also suitable because the CIC-IDS2017 features are tabular flow statistics. Decision Trees can capture non-linear feature interactions and threshold-based decisions, such as unusual packet lengths or flow rates. However, a single tree can overfit strongly to the training data. Random Forest reduces this risk by training many decision trees on bootstrap samples and averaging their predictions. This makes it more stable and often effective on tabular data. Extra Trees are similar, but introduce more randomisation when choosing split thresholds, which can reduce variance but may also slightly reduce precision if the random splits are not optimal.

HistGradientBoosting was selected as the strongest candidate model because gradient boosting builds trees sequentially, where each new tree focuses on correcting the errors of the previous trees, following the general gradient boosting approach described by Friedman (2001). This can capture complex non-linear patterns while maintaining strong predictive performance on large tabular datasets. The histogram-based implementation is also efficient for large datasets, which was important because the final modelling dataset contained over 2.5 million records. The main disadvantage

is that boosted trees can still learn dataset-specific patterns if the train/test split is too similar, so robustness testing is needed.

The models therefore represent a useful comparison between a simple linear learner and more flexible non-linear tree ensembles. Logistic Regression provides a strong interpretable baseline, while Random Forest and HistGradientBoosting provide higher-capacity models that are theoretically well suited to tabular intrusion-detection data.

The validation results supported the theoretical model comparison. The Dummy Baseline achieved 0.831108 accuracy but an attack F1 of 0.000000, showing that accuracy alone was misleading because the baseline did not detect any ATTACK flows. Logistic Regression performed much better than the baseline, with attack recall of 0.975969 and attack F1 of 0.845843, but its lower attack precision showed that the linear model produced more false alerts. The tree-based models performed substantially better overall. HistGradientBoosting achieved the highest validation attack F1 of 0.997360, followed closely by Random Forest at 0.996711. This evidence supported selecting HistGradientBoosting and Random Forest for the focused hyperparameter-tuning stage.

Table 1. Validation comparison of candidate machine learning models.

Model	Accuracy	Macro F1	Attack Precision	Attack Recall	Attack F1	PR-AUC
HGB	0.999107	0.998411	0.996527	0.998194	0.997360	0.999252
Random Forest	0.998887	0.998020	0.994823	0.998605	0.996711	0.999808
Decision Tree	0.998567	0.997447	0.995874	0.995640	0.995757	0.992352
Extra Trees	0.998304	0.996978	0.995532	0.994422	0.994977	0.995951
Logistic Regression	0.939917	0.904265	0.746335	0.975969	0.845843	0.946511
Dummy Baseline	0.831108	0.453882	0.000000	0.000000	0.000000	0.168892

(Note: HGB = HistGradientBoosting)

These very high validation scores were interpreted cautiously rather than treated as proof of real-world deployment readiness. Since several tree-based models achieved near-perfect random-split performance, the validation results suggest that the random split is an easier in-distribution benchmark where similar capture-day and attack-scenario patterns may appear in both training and validation data. This does not invalidate the comparison, because all candidate models were evaluated under the same split, but it means the random-split results should be treated as an optimistic estimate. This was one reason for adding the held-out Friday experiment as a stricter robustness test.

3. Final Model Selection and Justification

Based on the validation and final random test results, the tuned HistGradientBoosting classifier was selected as the final in-distribution model. The task is a supervised binary classification problem because the model learns from labelled CIC-IDS2017 network flows where the target class is either BENIGN or ATTACK.

HistGradientBoosting is an ensemble tree-based method that builds decision trees sequentially. Each stage attempts to correct errors made by the earlier stages, allowing the model to learn complex non-linear relationships between network-flow features and the binary attack label.

HistGradientBoosting optimises log-loss, also known as binary cross-entropy in the binary classification setting, which is the loss used by Scikit-learn's HistGradientBoostingClassifier for probabilistic classification (scikit-learn, n.d.). This loss function is appropriate because the model outputs class probabilities rather than only hard labels. For an intrusion detection system, probability scores are useful because the classification threshold can be adjusted depending on the operational trade-off between missed attacks and false alerts.

This model was selected because it achieved the strongest overall performance on the validation set and remained the best model on the untouched random final test set. It was also computationally suitable for the dataset size because the histogram-based implementation is designed to handle large tabular datasets efficiently (scikit-learn, n.d.). This was important because the final modelling dataset contained more than 2.5 million rows after cleaning and deduplication.

Threshold optimisation was performed separately from hyperparameter tuning. After the best models were trained, different probability thresholds were evaluated on the validation set only. The threshold that maximised attack F1 was then frozen and applied once to the untouched final test set. This avoids using the test set to choose the threshold and therefore reduces evaluation leakage. For the tuned HistGradientBoosting model, the selected validation threshold was 0.47. Because this threshold is slightly below 0.50, the model is tuned to flag attacks more readily, which prioritises reducing missed attacks while still maintaining very high attack precision.

Although HistGradientBoosting was selected as the final random-split model, the held-out Friday experiment showed that it was not the most robust model under distribution shift. Logistic Regression achieved better attack F1 when trained on Monday–Thursday traffic and tested on Friday traffic. This suggests that the boosted tree model learned in-distribution patterns very effectively but may also have captured capture-day or attack-scenario-specific patterns. This is especially important because some major Friday attacks, including PortScan and DDoS, appear only in the Friday files. Therefore, the held-out Friday test partly evaluates unseen attack-scenario generalisation, not just a normal random sample from the same distribution. The final model is best interpreted as the strongest in-distribution detector, while the held-out Friday results provide a more cautious view of generalisation.

4. Model Design, Training and Evaluation Strategy

Before training, the original Label, binary_label and source_file columns were excluded from the feature set. The Label and binary_label columns were removed to prevent direct target leakage, while source_file was removed because it identifies the capture file rather than a genuine network-flow property. It was retained only for auditing the dataset and constructing the held-out Friday experiment. A target-leakage sanity check was also carried out by searching the final feature list for terms such as label, attack or class, and no target-derived feature names were found.

The deduplicated dataset was split into training, validation and final test sets using stratified sampling. Stratification was used so that the BENIGN and ATTACK proportions remained consistent across the splits. The final test set was kept untouched during model comparison, tuning and threshold selection. The split was approximately 64% training, 16% validation and 20% final testing. The training set was used to fit the models, the validation set was used for model comparison and threshold tuning, and the final test set was used only once for the final random-split evaluation.

All preprocessing was placed inside a Scikit-learn Pipeline. A StandardScaler was applied to the numeric features through a ColumnTransformer. Although tree-based models do not strictly require feature scaling, using a common preprocessing pipeline ensured that Logistic Regression and tree-based models could be evaluated consistently. It also prevented preprocessing leakage because scaling parameters were learned only from the training data during fitting and from the training folds during cross-validation, rather than from the validation or final test data.

The initial validation comparison included a Dummy Classifier, Logistic Regression, Decision Tree, Random Forest, Extra Trees and HistGradientBoosting. The Dummy Classifier provided a lower-bound baseline, while the other models represented both simple linear learning and more complex non-linear tree-based methods.

HistGradientBoosting and Random Forest were then selected for hyperparameter tuning because they were the strongest tree-ensemble candidates from the validation comparison. They were tuned using GridSearchCV with stratified cross-validation on a 250,000-row subset taken from the training set only. A focused hyperparameter grid

was used because training on the full 2.5 million-row dataset was computationally expensive. The tuning subset preserved the original class balance while ensuring that validation and test data were not used during hyperparameter selection.

Evaluation used several metrics rather than relying only on accuracy. Accuracy was reported because it gives an overall success rate, but it is not sufficient for imbalanced intrusion detection. Macro F1 was used because it gives equal weight to the BENIGN and ATTACK classes. Attack precision, attack recall and attack F1 were reported separately because the operational cost of false positives and false negatives is different in cyber security. PR-AUC was also included because precision-recall analysis is especially informative for imbalanced binary classification, where the positive ATTACK class is the minority class (Saito and Rehmsmeier, 2015). Confusion matrices were used to examine the actual number of benign flows incorrectly flagged as attacks and attacks missed by the detector.

In addition to the random final test evaluation, a held-out Friday experiment was designed to test robustness. In this setting, all Monday–Thursday files were used for training, and the Friday files were used only for testing. This is a stricter evaluation because the Friday files contain different capture-day conditions and include attack scenarios such as DDoS and PortScan that are not present in the Monday–Thursday training data. This experiment therefore tests whether the models generalise beyond the random split, where similar capture-day patterns may appear in both training and testing.

Finally, the trained models were saved using joblib, and reload checks were performed to confirm reproducibility. The saved HistGradientBoosting model was reloaded and used to reproduce the final random test result. The saved Logistic Regression model from the held-out Friday experiment was also reloaded and checked against the held-out result. In both cases, the notebook confirmed that the reloaded models reproduced the reported test metrics to within a numerical tolerance of $1e-6$. This ensured that the submitted trained models could be loaded and used to run the tests reported in the coursework.

5. Results, Evaluation and Reflection

The random final test results show that the tuned HistGradientBoosting model performed extremely strongly under the main in-distribution evaluation. Using the validation-selected threshold of 0.47, it achieved an accuracy of 0.999155, macro F1 of 0.998496, attack precision of 0.996624, attack recall of 0.998379 and attack F1 of 0.997501. Its PR-AUC was 0.999878, showing that the model maintained a very strong precision-recall trade-off across thresholds. The tuned Random Forest model was also strong, but slightly weaker, with attack precision of 0.995698, attack recall of 0.997675 and attack F1 of 0.996686. Therefore, HistGradientBoosting was selected as the best random-split model. The corresponding precision-recall curve is shown in Figure 2.

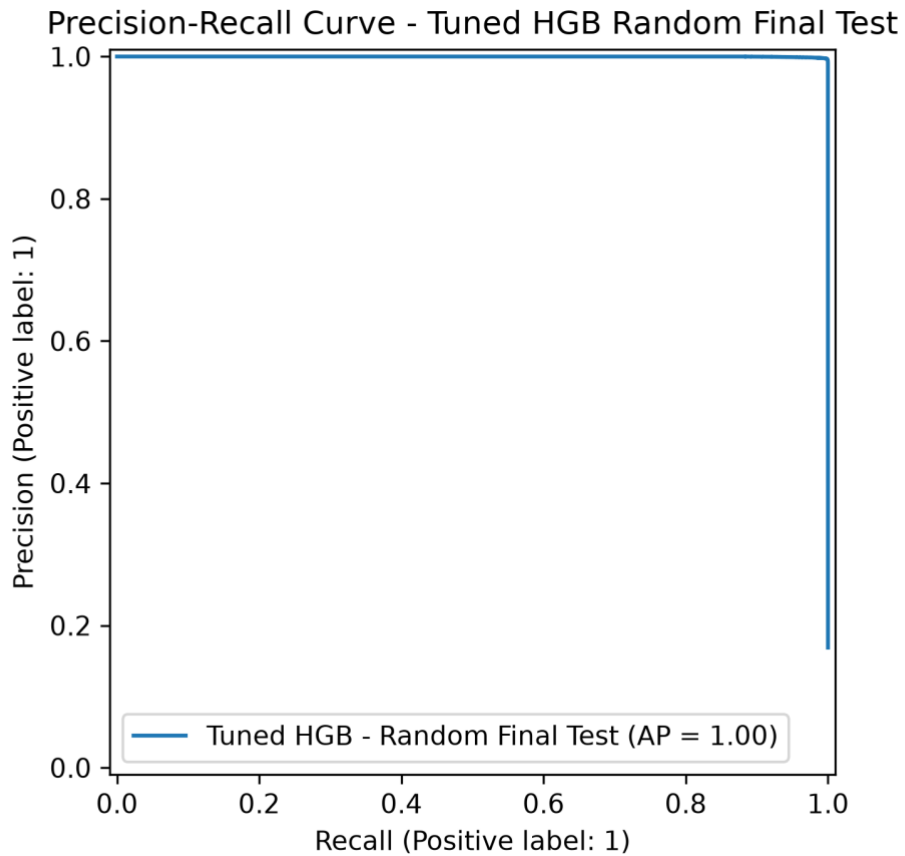


Figure 2. Precision-recall curve for tuned HistGradientBoosting on the random final test set.

The confusion matrix gives a more meaningful view than accuracy alone. On the 504,160-flow final test set, the HistGradientBoosting model correctly classified

418,724 BENIGN flows and 85,010 ATTACK flows. It produced 288 false positives and 138 false negatives, meaning that only 426 flows were misclassified in total. From an intrusion detection perspective, the 138 false negatives are the more serious error type because they represent attacks that would not be flagged by the detector. The 288 false positives are also operationally important because they could create unnecessary alerts for analysts. However, both error counts are very small relative to the test-set size, showing that the model performed very well when the test data followed the same broad distribution as the training and validation data. This is shown in Figure 3.

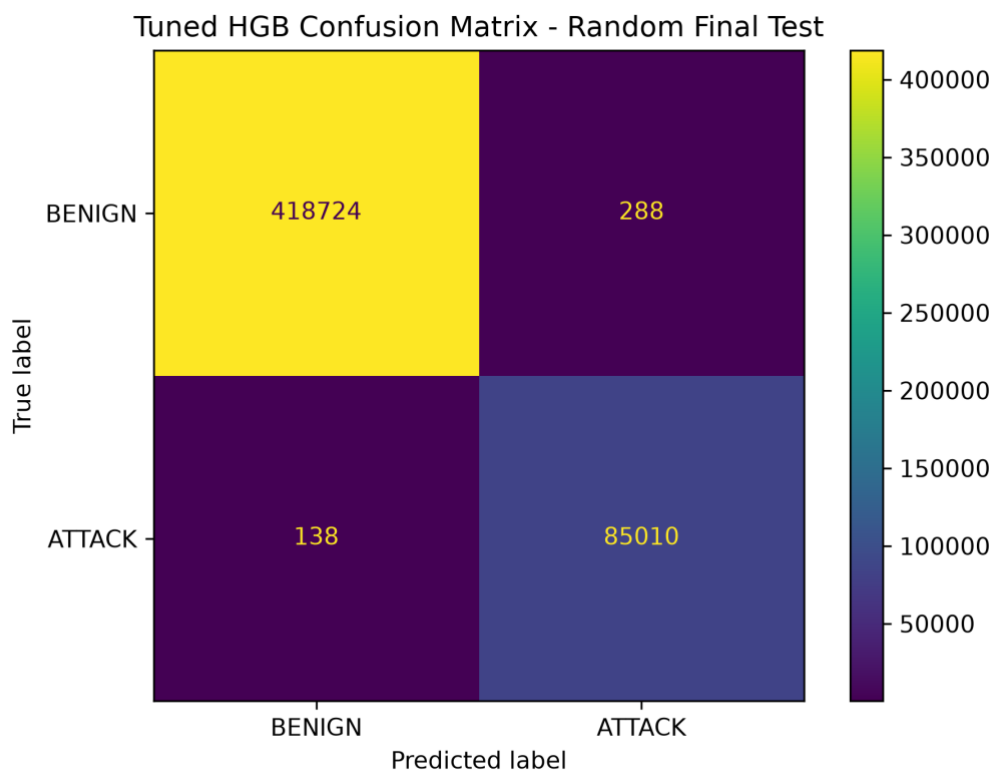


Figure 3. Confusion matrix for tuned HistGradientBoosting on the random final test set.

However, the held-out Friday experiment gives a more cautious view of model generalisation. When models were trained on Monday–Thursday traffic and tested only on Friday files, performance dropped substantially. Logistic Regression was the strongest model in this setting, achieving an attack F1 of 0.719816 at the default 0.50 threshold. Threshold sensitivity analysis found a best threshold of 0.51, but this only increased attack F1 slightly to 0.719959. This shows that the performance drop was

not mainly caused by poor threshold selection. Instead, it reflects a genuine generalisation challenge.

The held-out Friday confusion matrix reinforces this point. At the selected 0.51 threshold, Logistic Regression correctly classified 383,487 BENIGN flows and 130,458 ATTACK flows, but it also produced 11,290 false positives and 90,198 false negatives. The large number of false negatives means that many Friday attacks were missed. This shows that the main weakness under the stricter split was collapsed attack recall: the model became too conservative on unfamiliar Friday attack patterns, especially compared with its performance under the random split. This is a significant limitation for a real intrusion detection system, where missed attacks can be more damaging than false alarms. Although Logistic Regression generalised better than the tree-based models in this stricter test, its recall was still not sufficient for deployment without further improvement. This held-out Friday confusion matrix is shown in Figure 4.

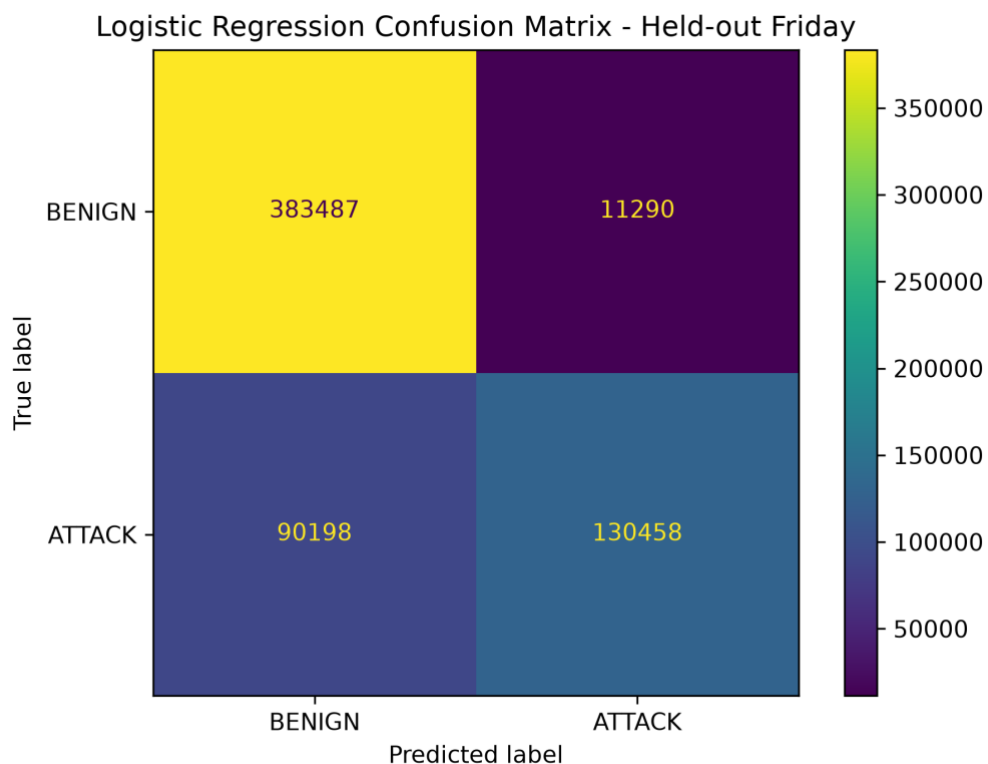


Figure 4. Confusion matrix for Logistic Regression on the held-out Friday test set.

The contrast between the random final test and the held-out Friday test is the most important finding of the project, and it is consistent with wider concerns that intrusion detection models can perform strongly on one dataset split but generalise less reliably under different traffic or dataset conditions (Cantone, Marrocco and Bria, 2024). The random split suggests that HistGradientBoosting can detect attacks almost perfectly when training and testing data share similar capture conditions. In contrast, the held-out Friday split shows that performance is much weaker when the model is tested on different capture-day and attack-scenario conditions. This is partly because major Friday attack types, including DDoS and PortScan, were absent from the Monday–Thursday training files. The held-out Friday result therefore tests unseen attack-scenario generalisation as well as day-level distribution shift. This explains why flexible tree ensembles, which performed extremely well in the random split, did not generalise as well as Logistic Regression in the stricter setting. The weaker precision-recall curve in Figure 5 further illustrates the generalisation gap.

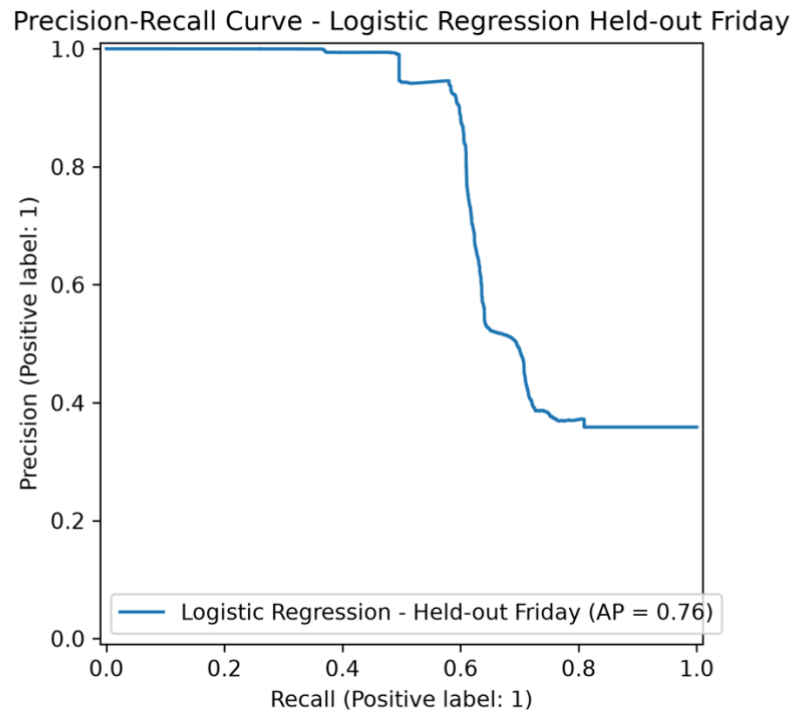


Figure 5. Precision-recall curve for Logistic Regression on the held-out Friday test set.

The feature-importance analysis for Random Forest also supports this interpretation. The most important features included Max Packet Length, Packet Length Std, Packet Length Variance, Average Packet Size, Avg Bwd Segment Size, Bwd Packet Length

Max, Destination Port and TCP window-size features. These are meaningful network-flow indicators because attack traffic often changes packet sizes, flow structure, port usage and packet-rate behaviour. However, these features may also encode attack-scenario-specific or capture-day-specific patterns. This is particularly relevant because Destination Port is closely linked to PortScan-style behaviour, while packet-length and packet-variance features are likely to separate high-volume attacks such as DDoS from benign traffic. Since PortScan and DDoS appear in the held-out Friday files rather than the Monday–Thursday training files, reliance on these discriminative random-split features helps explain why the model performed strongly in-distribution but weakened under the held-out Friday test. This means that strong performance on a random split does not necessarily prove that the model has learned general intrusion behaviour that will transfer to future attacks. The top-ranked features are shown in Figure 6.

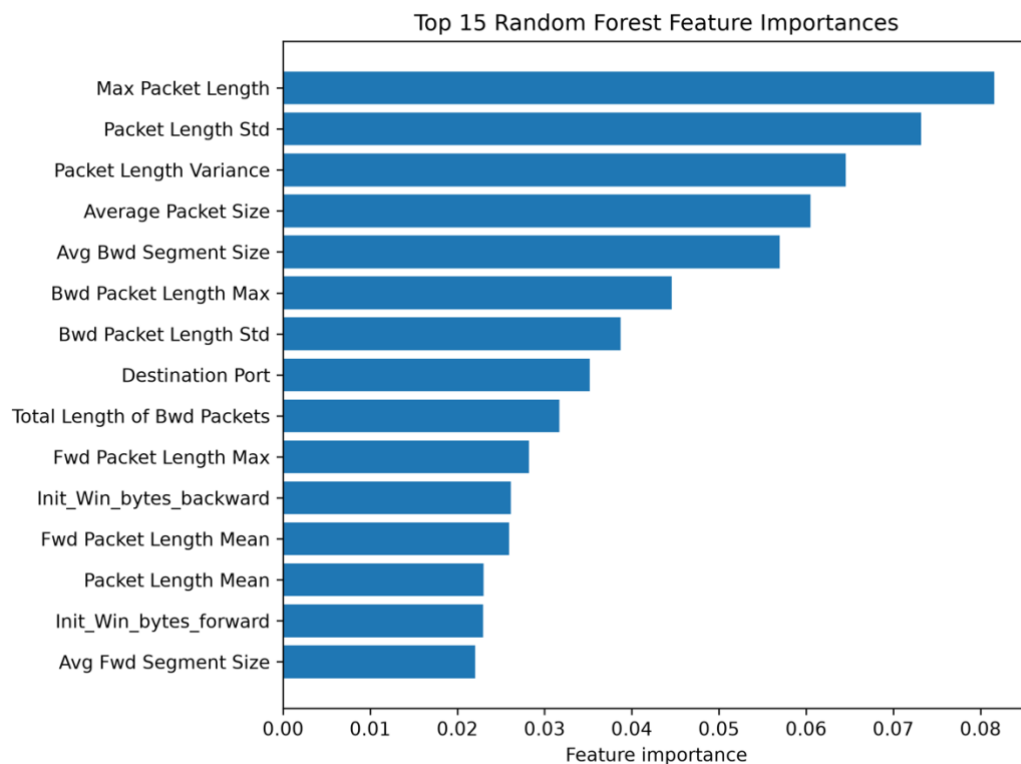


Figure 6. Top 15 Random Forest feature importances.

Overall, the solution met the main coursework objective by building, tuning, saving and evaluating machine learning models for binary intrusion detection. The in-distribution results met or exceeded expectations, because the tuned

HistGradientBoosting model achieved very high attack precision and recall on the untouched random final test set. The held-out Friday results were more cautious, but this was expected because the experiment was deliberately designed as a stress test for generalisation. The gap between the two evaluations is therefore not simply a weakness of the coursework solution; it is an important finding about how intrusion detection models can appear stronger under random splits than under more realistic distribution-shift conditions, which is consistent with prior work on dataset quality and generalisation challenges in network intrusion detection (Lanvin et al., 2023; Cantone, Marrocco and Bria, 2024).

The main limitation of this project is that the binary formulation detects malicious traffic but does not identify the specific attack type. This was a deliberate design choice because of the severe class imbalance in the original labels, but it limits the practical usefulness of the model for incident response. A second limitation is that the held-out Friday experiment is strict but imperfect: it combines capture-day shift with unseen attack-type shift, so it cannot isolate the exact cause of the performance drop. Future work should evaluate group-aware or time-aware cross-validation, multiclass or hierarchical classification with rare-class handling, and testing on external datasets. Further work could also examine calibration and cost-sensitive threshold selection, where the threshold is chosen based on the operational cost of missed attacks versus false alerts.

Code and model repository:

https://github.com/tvishaprasad/AICS_CICIDS2017_Project.git

6. References

Sharafaldin, I., Habibi Lashkari, A. and Ghorbani, A.A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. [online] doi:<https://doi.org/10.5220/0006639801080116>.

Canadian Institute for Cybersecurity (2017). *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. [online] [Www.unb.ca](http://www.unb.ca). Available at: <https://www.unb.ca/cic/datasets/ids-2017.html>.

Friedman, J.H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), pp.1189–1232. doi:<https://doi.org/10.1214/aos/1013203451>.

Maxime Lanvin, Gimenez, P.-F., Han, Y., Frédéric Majorczyk, Ludovic Mé and Éric Totel (2023). Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. *Lecture notes in computer science*, pp.18–33. doi:https://doi.org/10.1007/978-3-031-31108-6_2.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2018). Scikit-learn: Machine Learning in Python. *arXiv:1201.0490 [cs]*. [online] Available at: <https://arxiv.org/abs/1201.0490>.

scikit-learn. (n.d.). *sklearn.ensemble.HistGradientBoostingClassifier*. [online] Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>.

Saito, T. and Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced

Datasets. *PLOS ONE*, [online] 10(3), p.e0118432.
doi:<https://doi.org/10.1371/journal.pone.0118432>.

Cantone, M., Marrocco, C. and Bria, A. (2024). Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study. *IEEE Access*, [online] pp.1–1. doi:<https://doi.org/10.1109/access.2024.3472907>.

7. Appendix A: AI Declaration

A.1 Description and justification of AI tool use

I used AI tools, mainly ChatGPT, as an allowed AI collaboration tool during this coursework. The AI tool was used to support planning, code debugging, report structuring, wording refinement and critical review. It was not used to replace my own implementation or judgement. I ran the Python notebook myself, checked the outputs, verified the reported values against my own notebook results, and made final decisions about the task framing, model selection, figures, references and interpretation.

AI support was used in the following ways:

- To help plan the structure of the machine learning workflow, including data cleaning, binary label creation, train/validation/test splitting, model comparison, hyperparameter tuning, threshold tuning and final evaluation.
- To help debug Python and notebook issues, such as file paths, saved output files, model reload checks and plotting figures.
- To help interpret model results, especially the difference between the random final test results and the held-out Friday generalisation experiment.
- To help refine the written report so that it clearly addressed the coursework brief, including problem analysis, model comparison, final model justification, evaluation and limitations.
- To help check whether claims in the report were supported by notebook outputs, such as class counts, confusion matrices, F1-scores and PR-AUC values.

A.2 Representative prompts used

The following list summarises the main types of prompts used during the coursework process. It is representative of the AI support used for planning, debugging, report drafting, checking and critical review.

- “Help me structure the report according to the coursework brief.”
- “Is binary classification acceptable for CIC-IDS2017 if my labels are BENIGN vs ATTACK?”
- “Help me justify why I grouped all non-BENIGN labels into ATTACK.”
- “Check whether these dataset counts and class imbalance numbers are suitable for the report.”
- “Help me write the section comparing Logistic Regression, Random Forest and HistGradientBoosting.”
- “Help me explain why accuracy alone is not enough for this imbalanced intrusion detection task.”
- “Help me interpret the confusion matrix with 288 false positives and 138 false negatives.”
- “Help me explain why the held-out Friday result is much lower than the random final test result.”
- “Help me decide where to place figures and captions in the report.”

A.3 Critical assessment of AI-generated output

All AI-generated suggestions were reviewed and edited before being included in the final submission. I checked numerical claims against my own notebook outputs before using them in the report. For example, class counts, duplicate counts, model metrics, thresholds and confusion-matrix values were verified from the executed notebook rather than accepted directly from AI output.

I also critically assessed the wording of the report to avoid overclaiming the results. In particular, the very high random-split performance was not presented as proof of real-world deployment readiness. Instead, I used the held-out Friday experiment to show that performance dropped under a stricter generalisation test. This interpretation was based on my model outputs and was refined through my own review of the results.

Some AI suggestions were modified or rejected when they were too broad, too confident, or not directly supported by the notebook. References were also checked and formatted separately rather than copied without review. The final report reflects my own implementation, outputs, evaluation choices and critical judgement, with AI used as a support tool for explanation, editing and review.